

Laboratorium algorytmy ewolucyjne

Zadanie 2 – sprawozdanie

Igor Piotrowiak, 325503

Treść

Napisać program rozwiązujący problem komiwojażera (minimalizacja drogi pomiędzy n miastami bez powtórzeń) przy pomocy algorytmu genetycznego. Zastosować reprodukcję przy użyciu nieproporcjonalnej ruletki, operator krzyżowania CX, oraz mutację równomierną.

Program powinien umożliwiać użycie różnych wielkości populacji, liczby iteracji, prawdopodobieństwa mutacji.

Program powinien zapewnić wizualizację wyników w postaci wykresów średniego, maksymalnego i minimalnego przystosowania (długości trasy) dla kolejnych populacji oraz 2 map (o wymiarach 10x10 punktów), na których będą wyświetlane miasta oraz drogi najdłuższa i najkrótsza.

Pokazać działanie programu na danych testowych składających się z 10 miast, opisanych za pomocą współrzędnych na mapie o wymiarach 10x10 punktów.

Dane testowe: miasta:

A(2,1), B(9,7), C(6,5), D(1,7), E(3,6), F(5,6), G(4,2), H(10, 4), I(7,3), J(8,10)

Instrukcja obsługi (Visual Studio 2022, Python)

Program implementuje algorytm genetyczny (GA) do rozwiązania problemu komiwojażera (TSP) dla przykładowego zestawu 10 miast. Celem jest znalezienie możliwie najkrótszej trasy odwiedzającej wszystkie miasta i wracającej do punktu startowego. Parametrami pracy programu można sterować poprzez zmienne globalne znajdujące się w sekcji `__main__`:

- `POP_SIZE` – liczba osobników w populacji
- `GENERATIONS` – liczba generacji
- `PM_GENE` – prawdopodobieństwo mutacji pojedynczego genu
- `ELITISM` – liczba najlepszych osobników kopiowanych bez zmian

```
POP_SIZE = 100
GENERATIONS = 300
PM_GENE = 0.02
ELITISM = 0
```

Dane nt. miast można zawrzeć w globalnych tabelach `CITY_NAMES` zawierającej nazwy miast oraz `CITIES`, zawierających informacje nt. położenia miast. Obie tabele znajdują się na górze programu.

```

CITY_NAMES = ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"]
CITIES = {
    "A": (2, 1),
    "B": (9, 7),
    "C": (6, 5),
    "D": (1, 7),
    "E": (3, 6),
    "F": (5, 6),
    "G": (4, 2),
    "H": (10, 4),
    "I": (7, 3),
    "J": (8, 10),
}

```

Aby uruchomić program w Visual Studio 2022, należy otworzyć plik .py, upewnić się, że środowisko ma skonfigurowanego interpretera Pythona, a następnie uruchomić projekt (Ctrl + F5).

Wyniki (najlepsze, średnie i minimalne dopasowanie w kolejnych generacjach) są wypisywane w konsoli, a po zakończeniu generowany jest wykres postępu i funkcji dopasowania, na konsoli zaś wypisywane są najgorsze i najlepsza trasa z ostatniego pokolenia wraz z ich długością.

Na wykresie postępu dopasowania:

- zielona linia przedstawia najgorsze dopasowanie w każdej generacji,
- niebieska – średnie dopasowanie populacji,
- pomarańczowa – najlepszy wynik.

Zbieżność linii oznacza stabilizację populacji i osiągnięcie optimum funkcji.

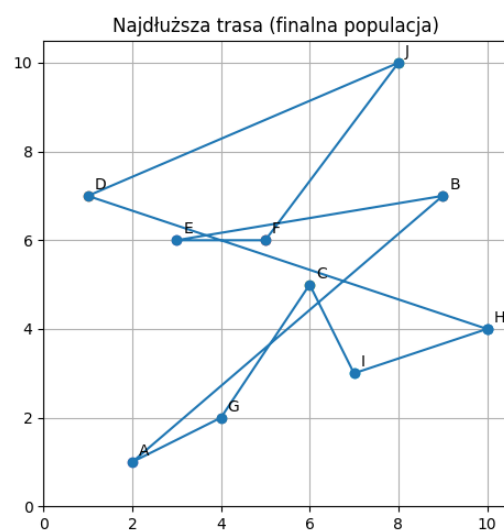
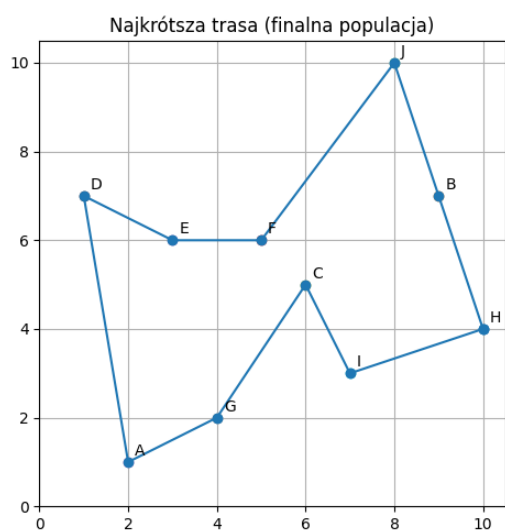
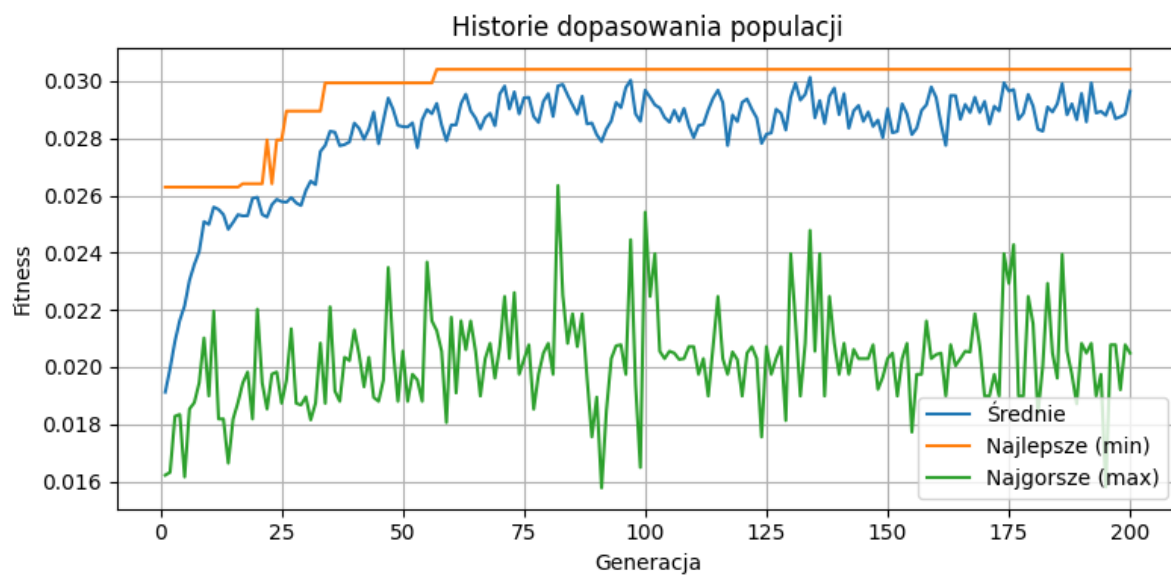
Program

Ze względu na prostotę implementacji, zdecydowałem się napisać swój projekt w języku Python, wykorzystując bibliotekę matplotlib do rysowania niezbędnych wykresów. Zgodnie z treścią polecenia, zawarłem w kodzie nieproporcjonalną ruletkę, krzyżowanie z użyciem operatora OX, a także mutację równomierną.

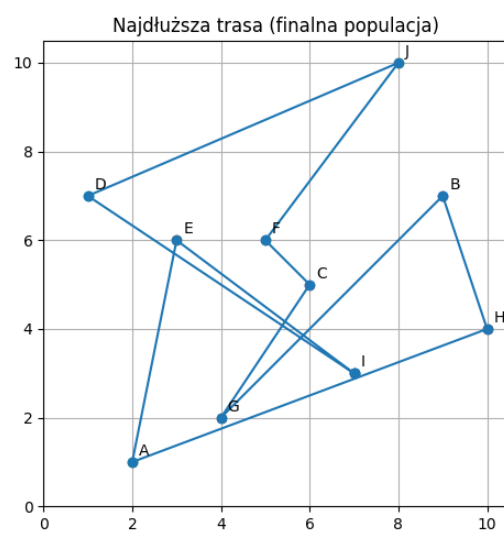
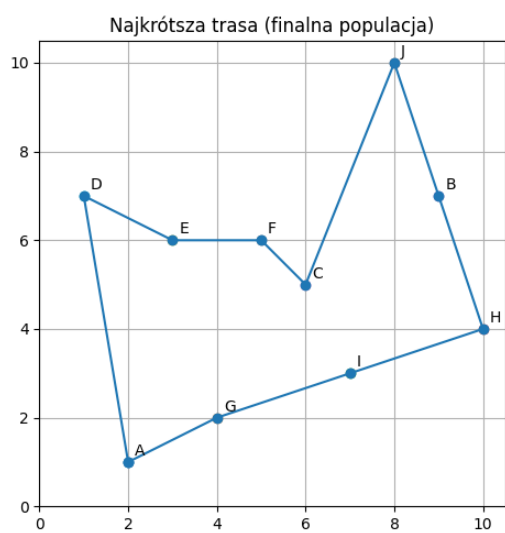
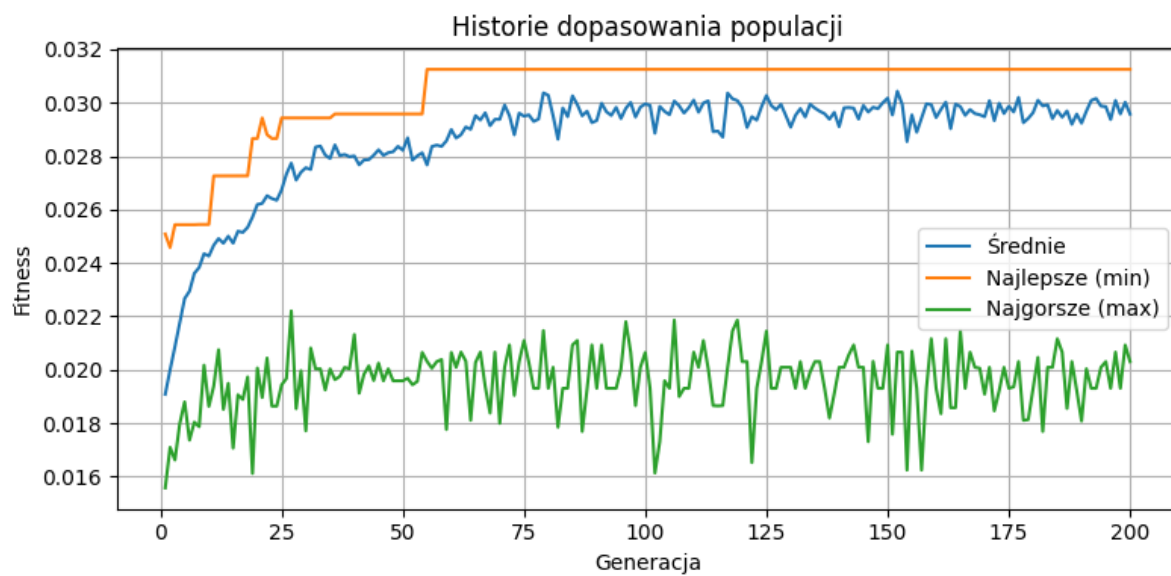
Badania

W celu zbadania zachowania algorytmu dla różnych parametrów zdecydowałem się przetestować go dla 3 różnych wielkości populacji (50, 100, 200), 3 parametrów mutacji (0.5%, 2%, 5%) i 3 różnych wartości maksymalnej liczby pokoleń (100, 200, 300). Domyślnymi wartościami, dla których będą przeprowadzane badania będzie wielkość populacji 100, prawdopodobieństwo mutacji 2% i ilość pokoleń 200.

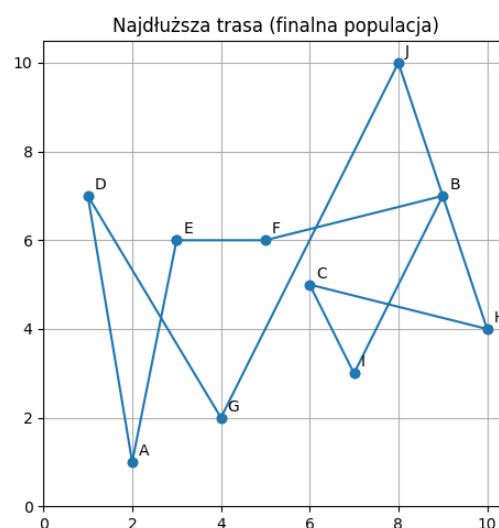
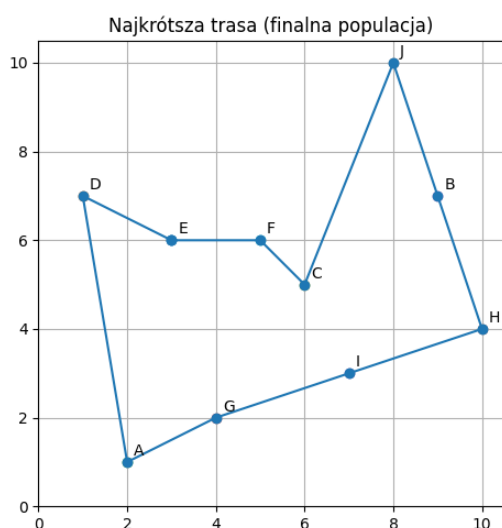
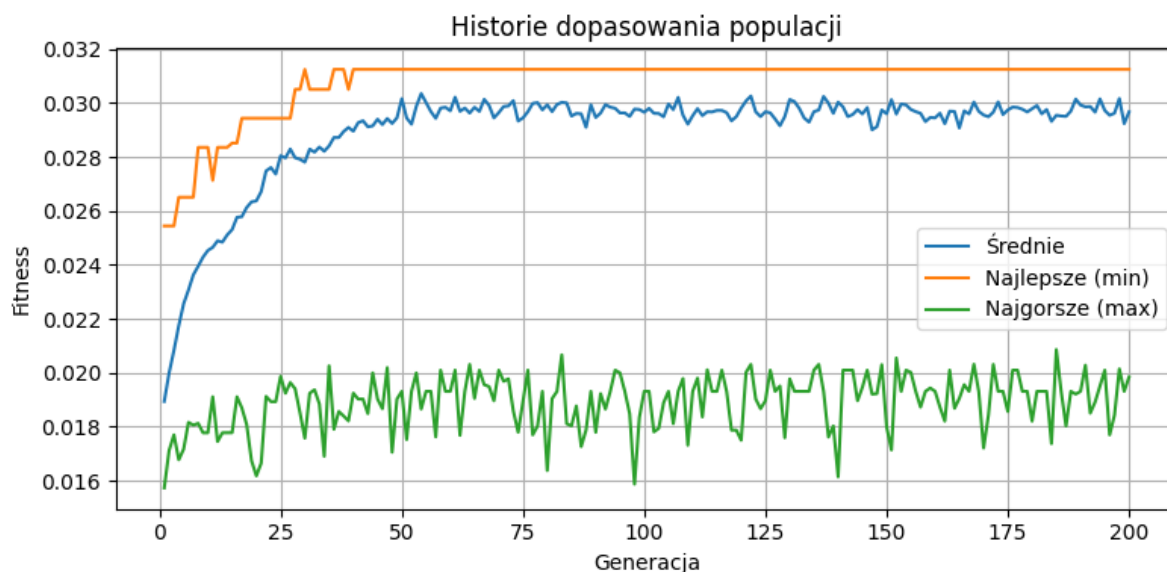
Porównanie wielkości populacji (prawd. mutacji 2%, ilość pokoleń 200)



Dla populacji wielkości 50



Dla populacji wielkości 100



Dla populacji wielkości 200

Jak widać na powyższych wykresach, każda próba charakteryzowała się nieco innym przebiegiem. Warto zauważyć, że tylko dwóm z nich udało się dojść do optymalnej trasy, którą jest 32,003 – udało się to w przypadku populacji liczącej 100 i 200 osobników. Próba z populacją 50 osobników zatrzymała się na nieoptymalnym rozwiązaniu 32,883 (algorytm prawdopodobnie utknął w minimum lokalnym).

Wraz ze zwiększaniem się liczby osobników w populacji można zauważyć stały trend – im większa liczba osobników, tym płynniejszy przebieg wykresu pokazującego średnie przystosowanie. Jest to zgodne z przewidywaniami – większa ilość osobników w populacji zapewnia mniejszy wpływ najlepszych i najgorszych rozwiązań.

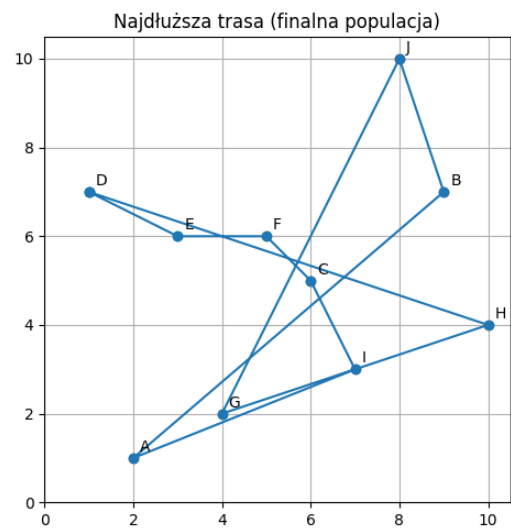
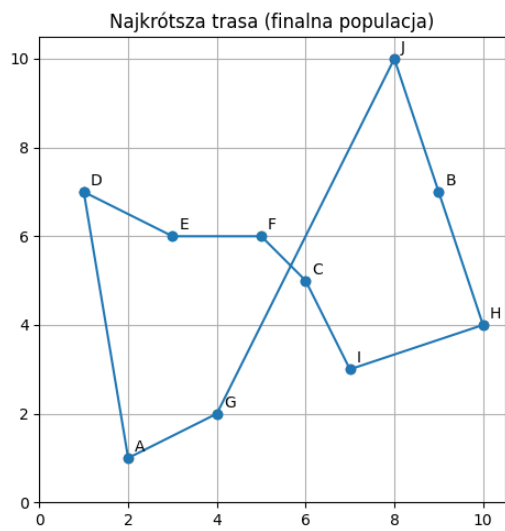
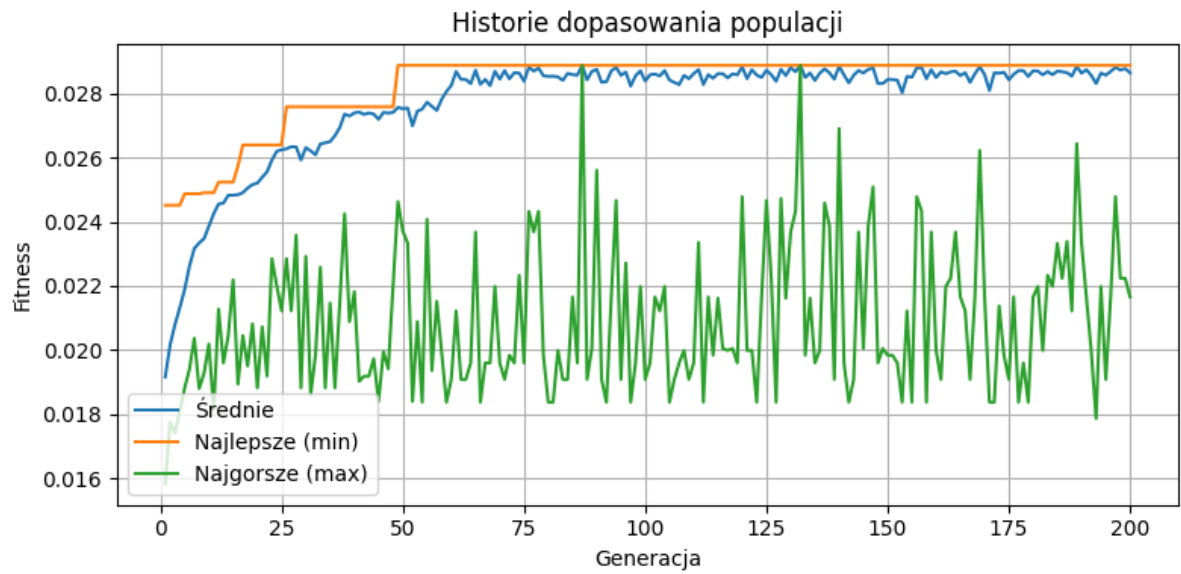
Dodatkowo, wraz ze zwiększaniem się liczby osobników widoczny jest spadek fluktuacji na wykresie przedstawiającym najgorszy wynik. Jego znaczne zmiany wartości są przede wszystkim widoczne na pierwszym wykresie prezentującym próbę dla 50 osobników. Poza tym moment osiągnięcia rozwiązania najbliższego optimum także się różni – dla prób z 50 oraz 100 osobnikami miało to miejsce w okolicy 55 generacji, natomiast dla próby ze 200 osobnikami znacznie wcześniej, w okolicach 40 pokolenia. Z tego można wyciągnąć wniosek, że wielkość populacji w pewnym zakresie może rekompensować ilość pokoleń, dla których przeprowadzono operacje.

Najgorsze wyniki osiągnięte w ostatnim pokoleniu prezentują się następująco: 50,645 dla 50, 53,723 dla 100 i 49,236 dla 200. Choć teoretycznie populacja wielkości 50 osiągnęła lepszy wynik od populacji 100, znaczna fluktuacja jej wykresu sugeruje, że mógł to być odosobniony przypadek. W ogólności, można się dopatrywać spadku najgorszego wyniku wraz ze zwiększaniem się liczby osobników, co także jest zgodne z oczekiwaniami.

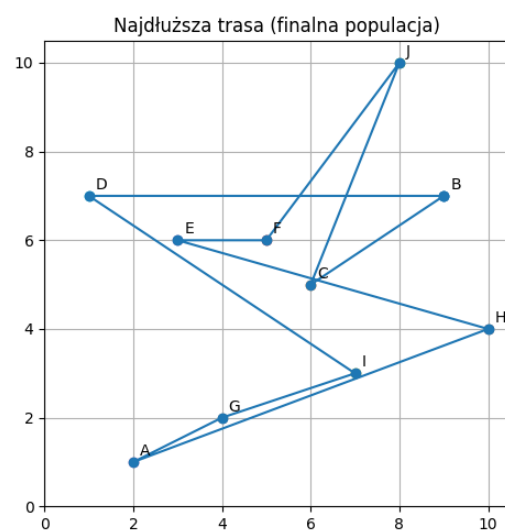
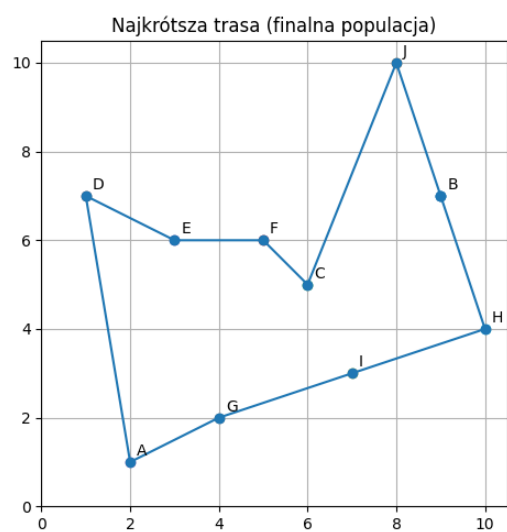
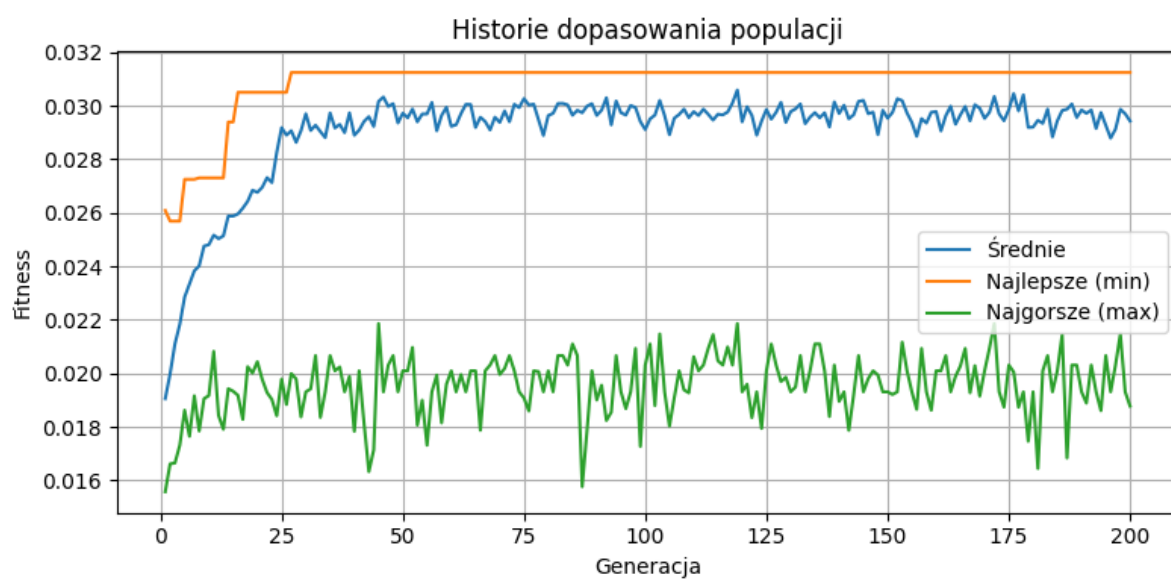
Podsumowując, Lepsze i stabilniejsze wyniki dla populacji 100 i 200 osobników wskazują, że większa populacja skuteczniej utrzymuje różnorodność genotypów, umożliwia eksplorację większej części przestrzeni rozwiązań, a także zmniejsza prawdopodobieństwo dominacji jednego schematu trasy we wczesnych generacjach. Dodatkowo, mniejsza ilość pokoleń potrzebna do osiągnięcia optymalnego rozwiązania w próbie z 200 osobnikami przy jednoczesnym wzroście koniecznych do przeprowadzenia obliczeń sugeruje istnienie kompromisu pomiędzy wielkością populacji a liczbą generacji, który w praktycznych zastosowaniach powinien być dobierany w zależności od dostępnych zasobów obliczeniowych. Spadek fluktuacji i tendencja do obniżania najgorszego wyniku wraz ze wzrostem populacji sugerują, że algorytm skuteczniej eliminuje bardzo słabe rozwiązania, a nawet „najgorsze” osobniki w dużej populacji reprezentują względnie sensowne trasy. Można to interpretować jako oznakę lepszego rozprzestrzeniania się dobrych cech w populacji.

Biorąc pod uwagę wspomniany wcześniej kompromis oraz między złożonością oraz ilością koniecznych pokoleń oraz otrzymane wyniki można stwierdzić, że wielkość populacji 100 w tym wypadku jest najlepsza.

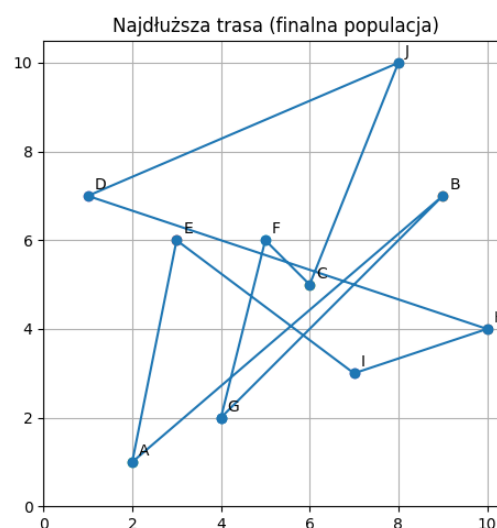
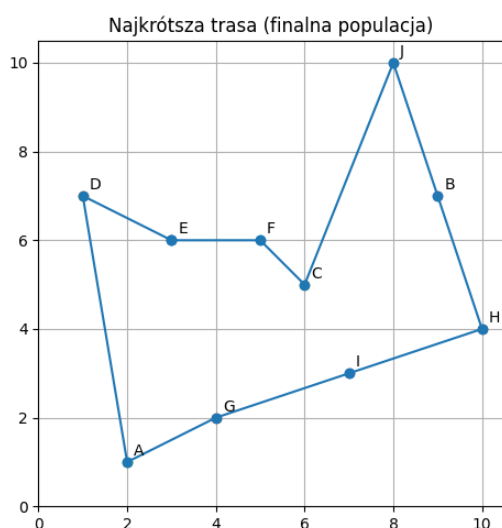
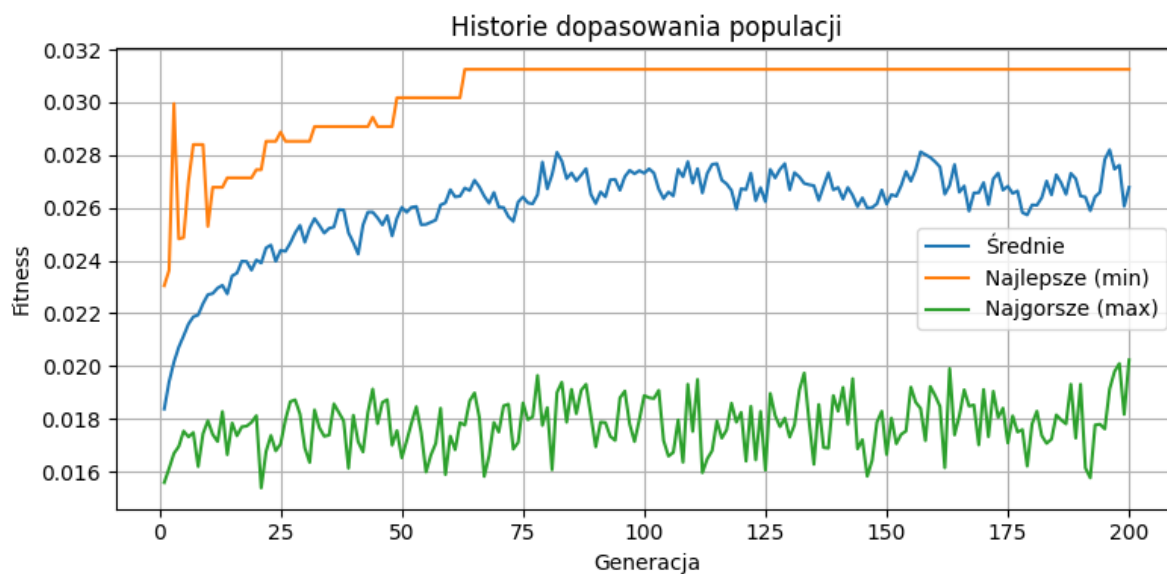
Porównanie prawdopodobieństwa mutacji (populacja 100, ilość pokoleń 200)



Dla prawdopodobieństwa mutacji 0,5%



Dla prawdopodobieństwa mutacji 2%



Dla prawdopodobieństwa mutacji 5%

Jak widać na powyższych wykresach, każda z prób charakteryzowała się zupełnie innym przebiegiem. Podobnie jak w poprzednim badaniu, optymalne wyniki (32,003) w ostatnim pokoleniu udało się osiągnąć dwóm próbom – dla prawdopodobieństwa mutacji 2% i 5%. Próba z prawdopodobieństwem 0,5% zatrzymała się na 34,636. Najgorsze wyniki w ostatnich pokoleniach dla prób 0,5%, 2% i 5% prezentowały się następująco: 50,409, 52,424 i 57,577.

Porównując każdy z wykresów można zauważyć kilka ciekawych elementów. Przede wszystkim to, że próba z prawd. 2% bardzo szybko osiągnęła optimum, którego się potem trzymała – stało się to już w okolicach 26 pokolenia. Dla próby 0,5% miało to miejsce ok. 49 generacji, a dla 5% znacznie później, ok. 62. Dodatkowo, interesującym jest, że dla prób 2% i 5% można zaobserwować regres najlepszego wyniku. O ile w przypadku próby 2% jest on bardzo niewielki i pojedynczy, o tyle dla próby 5% miał miejsce aż 4 razy, na różnych etapach przed osiągnięciem optimum i w pierwszych pokoleniach wahania te charakteryzowały się dość znacznymi wartościami. Prawdopodobnie zastosowanie elitaryzmu mogłoby temu zapobiec bądź chociaż zmniejszyć wielkość tych zmian.

Interesujące są przebiegi dla średnich wartości i pokrywają się one całkowicie z moimi wcześniejszymi podejrzeniami. Dla próby 0,5% wartość średnie dopasowanie gwałtownie rośnie, bardzo zbliżając się dla wartości najlepszej, dla dwóch pokoleń nawet ją osiągając. Po ustabilizowaniu

się najlepszego rozwiązania można zauważyć tylko niewielkie fluktuacje na wykresie. Dla próby 2% mamy podobnie gwałtowny spadek na początku i ustabilizowanie się po osiągnięciu wartości optymalnej; w przeciwieństwie do wcześniej opisanego przypadku, wartość średnia nigdy nie zbliża się aż tak bardzo do wartości najlepszej, ale warto zauważyć, że i tak osiąga lepsze przystosowanie niż w próbie 0,5%. W próbie 5% mamy do czynienia ze znacznie łagodniejszym spadkiem, a także wykres charakteryzuje się znacznie większymi fluktuacjami w porównaniu do dwóch poprzednich prób.

Natomiast wykresy najgorszych wartości całkowicie odbiegają od moich oczekiwań. W każdej próbie pojawiają się na nich znacznie większe fluktuacje w porównaniu do dwóch poprzednich, jednak interesującym jest, że wraz ze zwiększaniem prawdopodobieństwa mutacji, te fluktuacje stają się coraz mniejsze, podobnie jak spada ich przystosowanie. Zgodnie z tym trendem, największe zmiany można zaobserwować dla prawdopodobieństwa mutacji 0,5%, mniejsze dla prawdopodobieństwa 2%, a dla prawdopodobieństwa 5% są one stosunkowo najmniejsze.

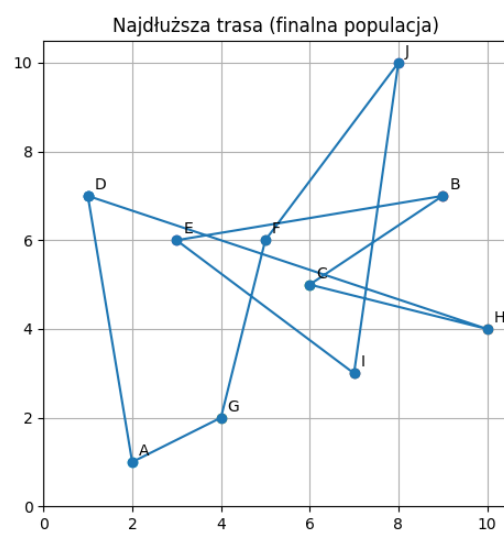
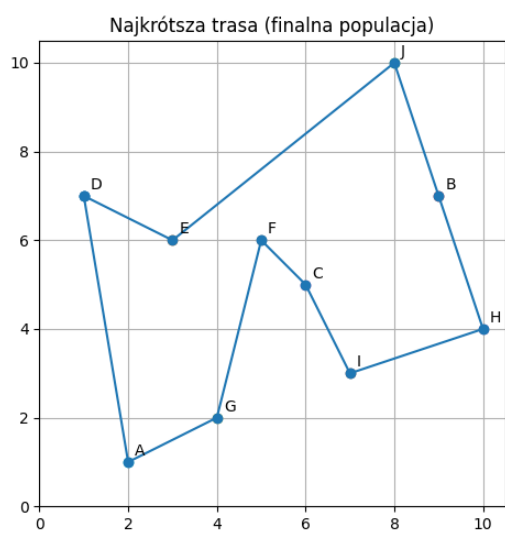
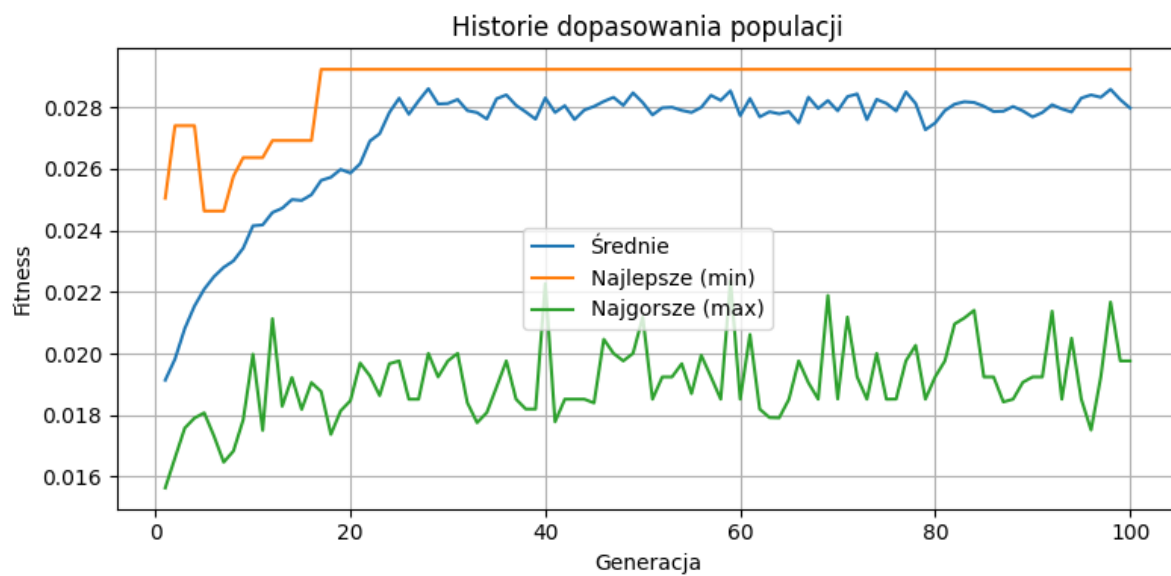
Podsumowując, przeprowadzone próby sugerują, że prawdopodobieństwo mutacji na poziomie około 2% stanowi najlepszy kompromis pomiędzy eksploracją a eksploatacją przestrzeni rozwiązań:

- Zbyt niska mutacja (0,5%) prowadzi do szybkiego ujednolicenia populacji i utraty różnorodności, co sprzyja utknięciu w minimum lokalnym.
- Zbyt wysoka mutacja (5%) powoduje nadmierną losowość, przez co dobre schematy są często niszczone, a proces zbieżności znacząco się wydłuża.

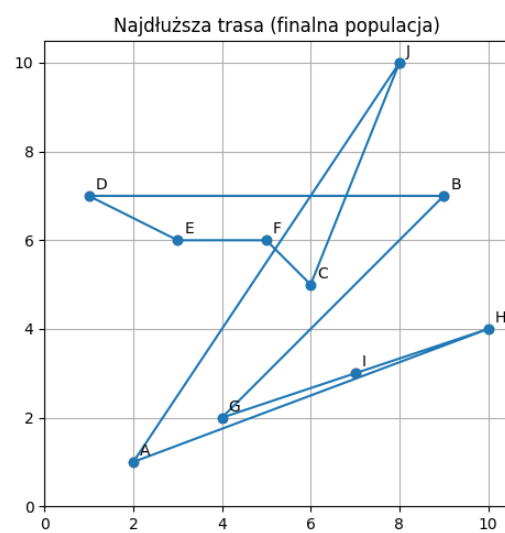
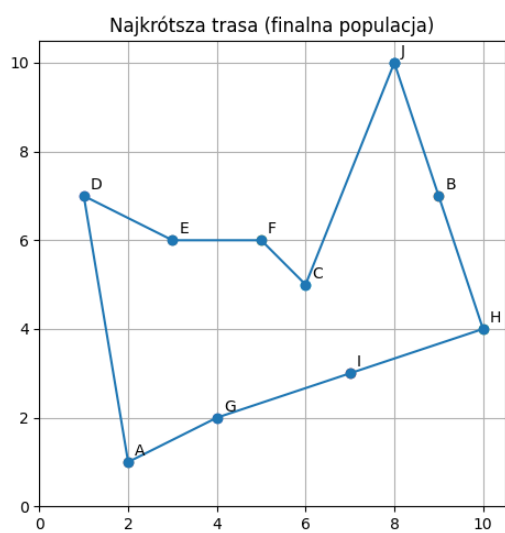
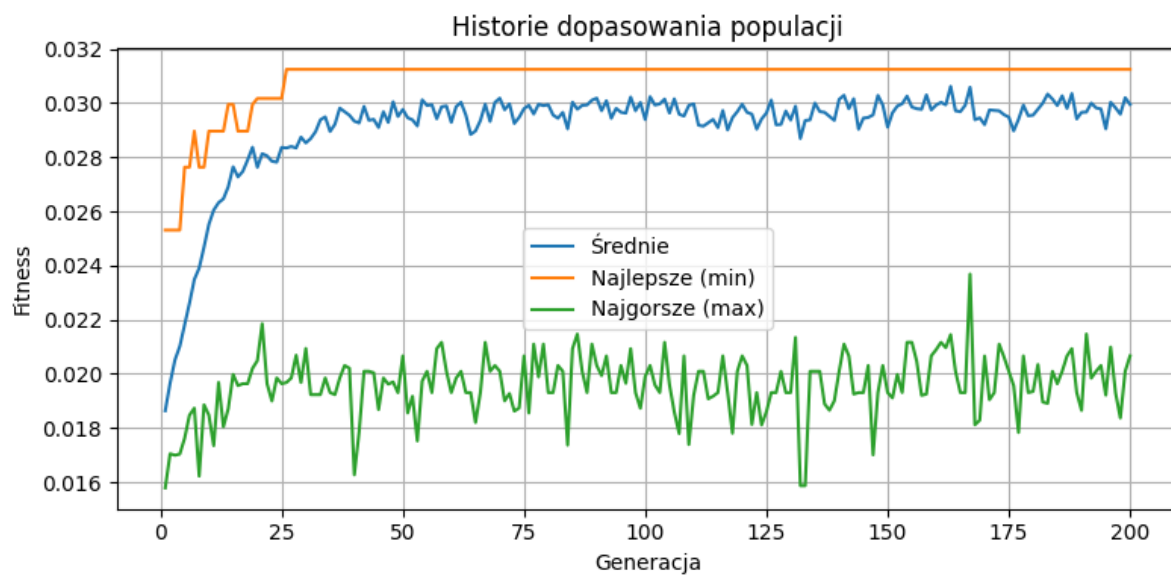
Dodatkowo, zgodnie z przewidywaniami, można zauważyć że wysoka mutacja znacznie obniża stabilizację. Zachowanie średniego dopasowania pokazuje, że niska mutacja sprzyja szybkiemu „sklejeniu się” populacji wokół jednego rozwiązania, średnia mutacja (2%) pozwala zachować różnorodność przy jednoczesnym wysokim poziomie jakości populacji, a wysoka mutacja powoduje stałe „rozmywanie” populacji, co objawia się łagodnym spadkiem i dużymi fluktuacjami średniego dopasowania.

Zmniejszanie się fluktuacji najgorszego dopasowania wraz ze wzrostem prawdopodobieństwa mutacji jest interesującym efektem i sugeruje, że częstsza mutacja zapobiega utrzymywaniu się skrajnie złych osobników.

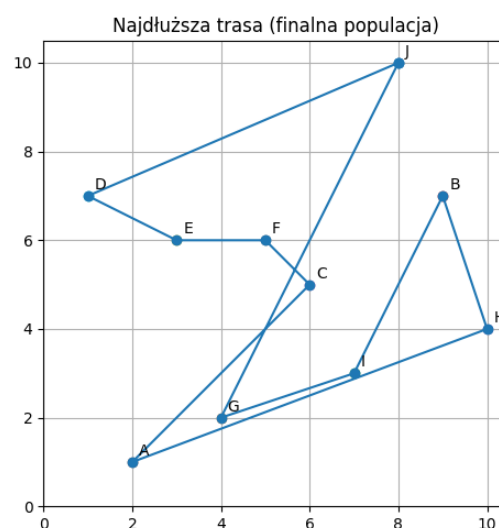
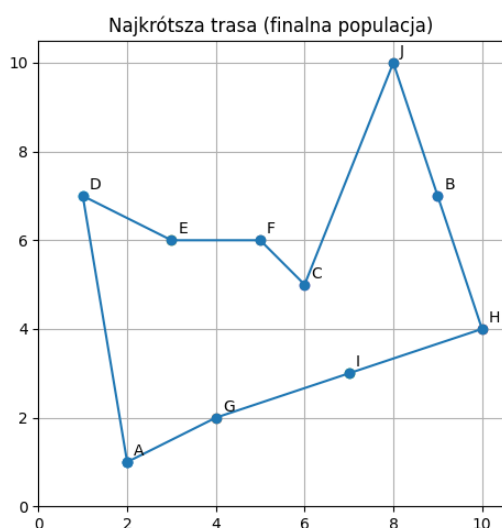
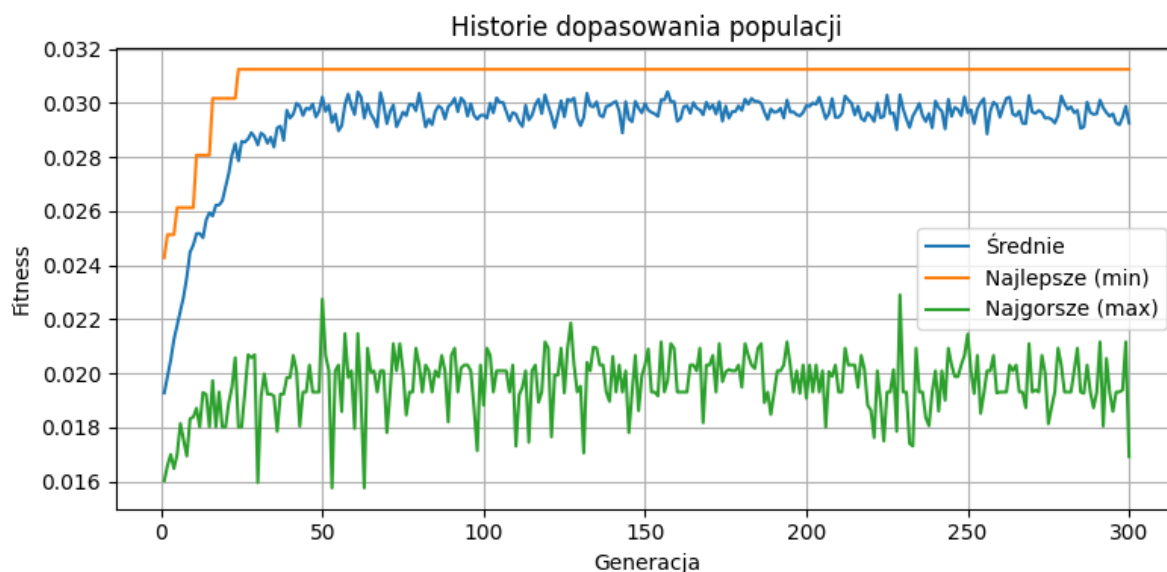
Porównanie ilości pokoleń (populacja 100, prawdopodobieństwo mutacji 2%)



100 pokoleń



200 pokoleń



300 pokoleń

W przeciwieństwie do dwóch poprzednich badań, w tym badaniu trudno się doszukać szczególnych różnic między poszczególnymi próbami. Kształty przebiegów nieco się różnią, ale wynika to prawdopodobnie wyłącznie z ich przeskalowania. Analogicznie do poprzednich badań, dwie próby osiągnęły optymalne rozwiązanie (32,003) – było to dla 200 i 300 pokoleń. Próba dla 100 pokoleń doszła tylko do 34,218. Prawdopodobnie nie jest to powiązane jednak bezpośrednio z liczbą pokoleń, jako że najlepszy wynik ustabilizował się znacznie przed dojściem do setnego pokolenia (miało to miejsce w okolicy 17 generacji)

Jedyna istotna różnica jest w wartościach dla najgorszych tras. Odpowiednio dla 100, 200 i 300 pokoleń było to 52,811, 51,792 o 47,208. Stosunkowo niewielkie fluktuacje dla wykresów najgorszych rozwiązań sugerują że jest to stały trend zależący wprost od liczby pokoleń, co jest zgodne z przewidywaniami.

Przebiegi wykresów wartości średnich dla wszystkich prób są bardzo zbliżone, a żadna z nich nie osiągnęła wartości najbliższej minimum dalej niż 30 pokolenie.

Podsumowując, liczba pokoleń nie wydaje się mieć kluczowego wpływu na zdolność algorytmu do odnalezienia rozwiązania optymalnego, o ile proces ewolucji osiąga stabilizację na wczesnym etapie.

W badanym przypadku najlepsze rozwiązania pojawiały się już w pierwszych kilkudziesięciu generacjach, a dalsze zwiększanie liczby pokoleń nie prowadziło do istotnej poprawy jakości najlepszego osobnika, a jedynie do stopniowego „oczyszczania” populacji z bardzo słabych rozwiązań.

Można zatem wnioskować, że dodatkowe pokolenia pełnią głównie rolę porządkującą populację, zmniejszając rozrzut wyników i poprawiając jakość najgorszych osobników, zamiast realnie wpływać na znalezienie lepszego optimum. Z praktycznego punktu widzenia oznacza to, że dalsze zwiększanie liczby pokoleń po osiągnięciu stabilizacji prowadzi do malejących korzyści przy rosnącym koszcie obliczeniowym.

W konsekwencji, w rozpatrywanym problemie bardziej opłacalne wydaje się zatrzymanie algorytmu po wykryciu stagnacji najlepszego rozwiązania lub zastosowanie adaptacyjnego kryterium stopu, zamiast arbitralnego zwiększania liczby pokoleń. Takie podejście pozwoliłoby skrócić czas obliczeń bez pogorszenia jakości uzyskiwanych rozwiązań.

Kierując się tą logiką, najlepszym rozwiązaniem spośród badanych byłoby 100 pokoleń, ponieważ ogranicza ono znacznie ilość koniecznego do przeznaczenia czasu.

Podsumowanie

Na podstawie otrzymanych wykresów oraz płynących z nich wniosków, można się pokusić o stwierdzenie, że parametrami, które pozwolą najbardziej zbliżyć się do optimum dla podanych warunków zadania jest wielkość populacji 100, prawdopodobieństwo mutacji 2% i ilość pokoleń w okolicach 50. Parametrem, który bez wątpienia okazał się mieć największy wpływ na otrzymane wyniki było prawdopodobieństwo mutacji, co sugeruje, że operator krzyżowania CX jest na nie bardzo wrażliwy.